Pandas

By PrudviKrishna

Pandas

- Pandas name is derived from the term **"Panel Data”**
- Pandas is an open-source data analysis and data manipulation library

- Pandas will provide two data structures
  - **Series:** A 1D labeled (or index) array capable of holding any data type.
  - **DataFrame:** A 2D labeled data structure, similar to a spreadsheet or SQL table, where data is organized in rows and columns.

#Importing Pandas library

```
import pandas as pd
```

Series

#Creating an Empty Series Object
```
empty_series = pd.Series()
print(empty_series)
```

#Creating Series from Lists/Arrays
```
data_list = [1, 2, 3, 4, 5]
series_from_list = pd.Series(data_list)
print(series_from_list)
```

#Creating Series with labels
```
import pandas as pd

data = [10, 20, 30, 40, 50]
labels = ['A', 'B', 'C', 'D', 'E']
series_with_labels = pd.Series(data, index=labels)
```

#Indexing and Accessing Values
```
print(series_from_list[1])
print(series_from_list['B'])
```

#Accessing series elements with slicing
```
print(series_from_list[1:4])
print(series_from_list['B':'D'])
```

#Use iloc for integer-location based indexing
```
print(series_from_list.iloc[0:3])
```

#Filter the series
```
print(series_from_list[series_from_list > 33])
```

 # To delete element form series
```
series_dropped = series.drop('B')
del series['D']
```

 # Inserting a new value
```
series['F'] = 60
```

 # Checking for NaN values
```
print(series_with_nan.isna()) # Returns a Boolean Series

series_from_list.size   #series Attributes
series_from_list.nunique() # series methods
```

DataFrame

```
# Creating a DataFrame from a dictionary

data_dict = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Los Angeles', 'Chicago']
}

df = pd.DataFrame(data_dict)
print(df)
```

Create DataFrame from Different Files

You can create a DataFrame by loading data from CSV, Excel, or JSON files.

```
# From a CSV file
df_csv = pd.read_csv('data.csv')
print(df_csv)

# From an Excel file
df_excel = pd.read_excel('data.xlsx‘)
print(df_excel)

# From a JSON file
df_json = pd.read_json('data.json')
print(df_json)
```

DataFrame Attributes and Methods

```
print(df.shape)  # Returns (rows, columns)
print(df.columns)  # Returns column names
print(df.index)  # Returns the row indices
print(df.dtypes)  # Returns data types of each column
```

```
print(df.head())  # Shows the first 5 rows
print(df.tail())  # Shows the last 5 rows
print(df.describe())  # Summary statistics
print(df.info())  # General info about the DataFrame
```

Rename Column & Index

 # Renaming columns

```
df_renamed = df.rename(columns={'Name': 'Full Name', 'Age': 'Years'})
print(df_renamed)
```

 # Renaming index

```
df_renamed_index = df.rename(index={0: 'ID_1', 1: 'ID_2'})
print(df_renamed_index)
```

Handling missing or NaN values

```
# Create a DataFrame with missing values
data_with_nan = {'Name': ['Alice', 'Bob', 'Charlie'],
                 'Age': [25, None, 35],
                 'City': ['New York', None, 'Chicago']}
df_nan = pd.DataFrame(data_with_nan)

# Filling NaN values
df_nan_filled = df_nan.fillna('Unknown')
print(df_nan_filled)

# Dropping rows with NaN values
df_nan_dropped = df_nan.dropna()
print(df_nan_dropped)
```

Slicing

Syntax: df.loc[start:end:step, start:end:step]
      df.iloc[start:end:step, start:end:step]

 # Slicing with loc (label-based)
print(df.loc[0]) # Select first row by label
print(df.loc[0:1, ['Name', 'Age']]) # Select specific rows and columns by label

 # Slicing with iloc (index-based)
print(df.iloc[0]) # Select first row by index
print(df.iloc[0:2, 0:2]) # Select specific rows and columns by index

DataFrame – Filtering, Sorting, GroupBy

```
# Filter rows where Age > 30
filtered_df = df[df['Age'] > 30]
#filtered_df = df[df["Age"] <30]["Name"]
print(filtered_df)

filtered_df = df[(df['Age'] > 30) & (df['Score'] > 90)]
print(filtered_df)

# Sorting by Age in descending order
sorted_df = df.sort_values(by='Age', ascending=False)
print(sorted_df)

# Group by City and calculate the mean Age
grouped_df = df.groupby('City')['Age'].mean()
#df.groupby(['City', 'Gender'])['Age'].mean()
print(grouped_df)
```

Merging or joining DataFrame

You can merge or join two DataFrames using keys.

```
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['Alice', 'Bob', 'Charlie']})
df2 = pd.DataFrame({'ID': [1, 2, 4], 'Score': [85, 90, 78]})

# Merge on 'ID' column
merged_df = pd.merge(df1, df2, on='ID', how='inner')  # Options: 'inner', 'outer', 'left', 'right'
print(merged_df)
```

Drop Rows & Columns

You can drop rows and columns using drop()
Axis will specify the operation to be performed on rows (axis=0) or columns (axis=1).

 # Dropping a row using axis=0 (default)

```
df_dropped_row = df.drop(0, axis=0)  # Here, we drop the row with index 0
print(df_dropped_row)
```

 # Dropping a column using axis=1

```
df_dropped_col = df.drop('Salary', axis=1)
print(df_dropped_col)
```

Aggregation Functions

sum, mean, count, etc.:
df.sum(axis=0) (default) calculates the sum for each column across all rows.
df.sum(axis=1) calculates the sum for each row across all columns.

Applying Functions:

- **apply Method**:
  - df.apply(func, axis=0) applies the function to each column.
  - df.apply(func, axis=1) applies the function to each row.

 # Summing across rows (result for each column)
column_sums = df.sum(axis=0)

 # Summing across columns (result for each row)
row_sums = df.sum(axis=1)

 # Applying a function to each column
result_columns = df.apply(lambda x: x.sum(), axis=0)

 # Applying a function to each row
result_rows = df.apply(lambda x: x.sum(), axis=1)

Concatenate Multiple CSV Files

```
import glob

# Get all CSV files in a directory
csv_files = glob.glob("*.csv")

# Read each CSV file and concatenate them
dfs = [pd.read_csv(file) for file in csv_files]
combined_df = pd.concat(dfs, ignore_index=True)
#combined_df = pd.concat([df1, df2], ignore_index=True)
print(combined_df)
```

Thank you